

Taak 1 ISLER GABRIEL 0631137

Inleiding

Het doel is Dijkstra te versnellen van $O((m+n) \log n)$ naar $O(m + n \log n)$ door een Fibonacci heap te gebruiken. De winst zit in reschedule!: bij een binaire heap kost die $O(\log n)$, bij een Fibonacci heap geamortiseerd $O(1)$. Voor dichte grafen, waar dat aantal naar m loopt, telt het verschil. De code zit in drie bestanden: de heap, een priorityqueue erbovenop, en het algoritme.

Architectuur

fibheap.rkt bevat de heap zelf, met de records fib-node en heap. Roots en childs zitten in circulaire dubbel-gelinkte lijsten. Geëxporteerde functies: new, empty?, insert!, peek, delete!, touch-at!, node-key, node-value. Hulprocedures zoals merge-roots!, cut! en cascading-cut! blijven intern, omdat ze enkel zinvol zijn binnen de heap.

pqfib.rkt is een priorityqueue-ADT rond de heap, geïnspireerd op de interface van de queue in de a-d-folder. Een notify-callback geeft na elke enqueue! of reschedule! het heap-knooppunt terug, zodat Dijkstra per vertex kan bijhouden welk heap-knooppunt erbij hoort. De notify-parameter van serve! wordt om redenen van interface-uniformiteit aanvaard maar niet gebruikt, aangezien het verwijderde minimum geen verdere mapping vereist.

dijkstra.rkt gebruikt enkel de queue-interface en raakt de heap niet rechtstreeks aan. Drie vectoren houden de toestand bij: distances voor de huidige kortste afstand per knoop, how-to-reach voor de voorganger op dat pad, en pq-nodes voor de mapping van vertex naar het bijhorende heap-knooppunt. Die laatste wordt gevuld door de callback register-node!, die als notify wordt doorgegeven aan elke enqueue! en reschedule!. Het algoritme zelf is een klassieke Dijkstra-loop: telkens de dichtstbijzijnde knoop bedienen, zijn burens relaxeren, en bij verbetering de prioriteit in de wachtrij verlagen.

Ontwerpkeuzes

De implementatie maakt gebruik van een “lazy” structuur waarbij insert! in $O(1)$ wordt uitgevoerd zonder enige aanpassing aan de interne opbouw. Pas bij delete! worden de wortels per graad samengevoegd in een bucket-vector, wat de geamortiseerde tijdscomplexiteit laag houdt.

Om te garanderen dat delete! zijn $O(\log n)$ bovengrens behoudt, wordt gebruikgemaakt van cascading cuts in combinatie met een marked-flag. Zonder dit mechanisme zouden deelbomen kunnen ontaarden, waardoor de tijdcomplexiteitsgaranties niet langer zouden gelden. Als bovengrens voor max-degree wordt de formule $\lfloor \log_2 n \rfloor + 2$ gehanteerd.

Bij reschedule! is het noodzakelijk te weten welk heapknooppunt bij een gegeven vertex hoort. Zonder een expliciete mapping zou reschedule! de volledige heap moeten doorzoeken, wat de operatie $O(n)$ zou maken. Om dit te vermijden geeft de queue via de notify-callback na elke enqueue! en reschedule! het betrokken knooppunt terug, zodat Dijkstra dit kan opslaan in pq-nodes.

+inf.0 wordt gebruikt als sentinel. Door alle afstanden op oneindig te initialiseren is geen aparte visited?-flag nodig.

dijkstra.rkt importeert pqfib.rkt voor de wachtrij en a-d/graph/weighted/config voor de graphinterface. De heap is daarmee in principe vervangbaar zonder het algoritme aan te raken.

Gebruikte a-d bestanden

Er werd enkel a-d/graph/weighted/config gebruikt van de a-d-folder. Niets in a-d werd gewijzigd. De queue in pqfib.rkt is bewust geïnspireerd op a-d/priority-queue/modifiable-heap.rkt.